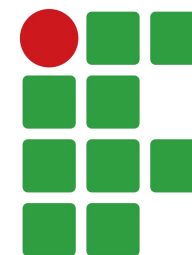


PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco

Gerenciadores de Layout

- Os gerenciadores de layout são componentes responsáveis por organizar, posicionar e dimensionar os elementos da interface do usuário em uma tela.
- Eles definem como as diferentes Views, como Button, TextView e ImageView, serão dispostas e dimensionadas dentro de uma Activity.
- Os gerenciadores de layout também podem conter outros layouts aninhados, formando uma hierarquia de elementos que define a estrutura da interface do usuário.

Gerenciadores de Layout

- Os elementos da interface são organizados por meio de uma hierarquia de objetos ViewGroup e View.
- Uma View representa um elemento da interface que pode exibir conteúdo e/ou responder à interação do usuário, como TextView, Button e ImageView.
- Um ViewGroup é um tipo especial de View que atua como um contêiner para outras Views e ViewGroups, sendo responsável por organizar seus elementos filhos de acordo com as regras definidas pelo layout.

Gerenciadores de Layout

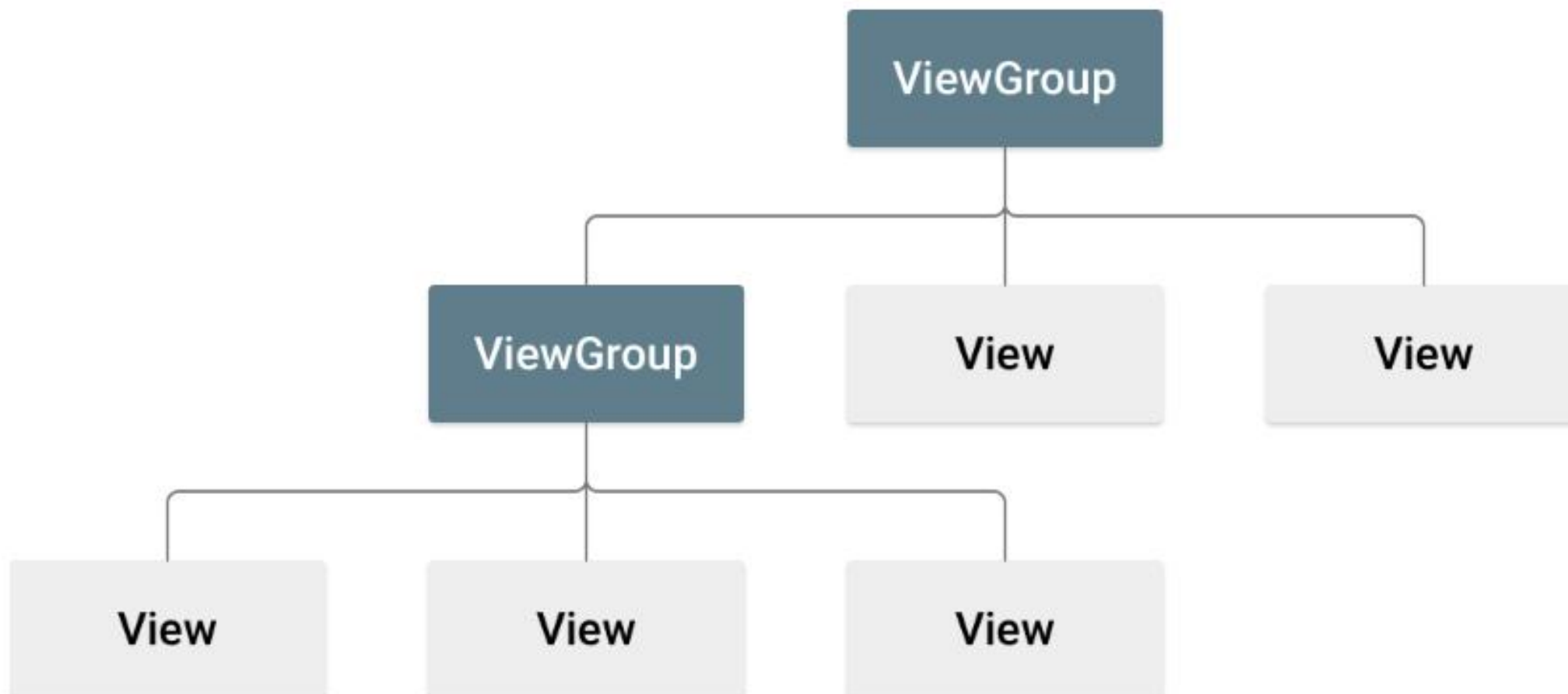


Figura 1. Ilustração de uma hierarquia de visualização, que define o layout da interface.

Gerenciadores de Layout

- Os objetos View representam elementos da interface, frequentemente chamados de widgets, como Button, EditText e TextView.
- Os objetos ViewGroup atuam como contêineres para organizar outras Views e ViewGroups.
 - Alguns tipos comuns de ViewGroup utilizados como layouts são LinearLayout e ConstraintLayout, cada um com diferentes regras para organizar seus elementos filhos.

Gerenciadores de Layout

- A interface de usuário pode ser definida de duas maneiras:
 - Por meio de arquivos XML, utilizando diretamente a linguagem XML ou o Layout Editor do Android Studio.
 - Programaticamente, instanciando e configurando as Views em tempo de execução.
- A definição da interface em XML permite separar a apresentação da interface do código responsável pelo comportamento da aplicação.
- O uso de arquivos XML também facilita a criação de diferentes versões da interface para diferentes tamanhos, densidades e orientações de tela.

Gerenciadores de Layout

- Cada arquivo de layout XML deve possuir exatamente um elemento raiz, que deve ser um objeto View ou ViewGroup.
 - A partir do elemento raiz, podem ser adicionados outros objetos View ou ViewGroup como elementos filhos, formando uma hierarquia que define a estrutura da interface.
- O arquivo de layout deve ser salvo com a extensão .xml no diretório res/layout/ do projeto Android.
- Os arquivos armazenados em res/layout/ são processados pelo sistema de recursos do Android e podem ser referenciados no código por meio da classe R ou, quando utilizado, por meio do View Binding.

Gerenciadores de Layout

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >
    <TextView android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a TextView" />
    <Button android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a Button" />
</LinearLayout>
```

Gerenciadores de Layout

- Durante a compilação, cada arquivo de layout XML é transformado em um recurso de layout que pode ser acessado por meio da classe R.
- Para exibir esse layout na Activity, o recurso deve ser carregado, normalmente, durante o método onCreate().
- O método setContentView() pode ser utilizado para definir o layout que será exibido pela Activity.

```
fun onCreate(savedInstanceState: Bundle) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.main_layout)  
}
```



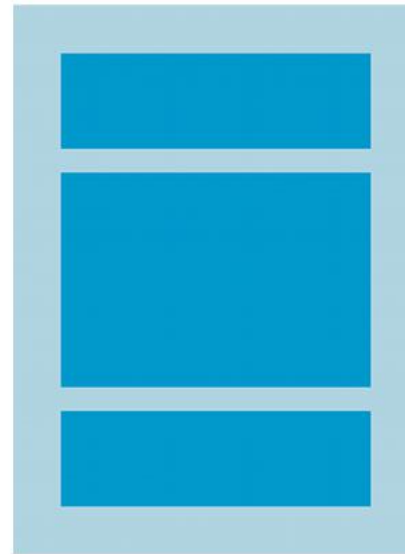
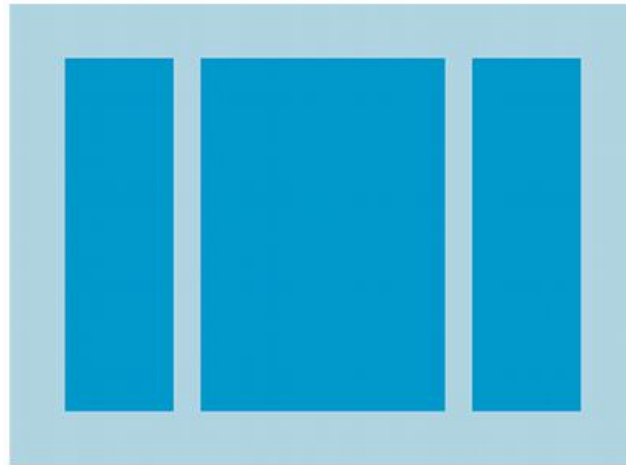
Nesse caso, R.layout.nome_do_layout representa o arquivo XML correspondente localizado no diretório res/layout/.

Gerenciadores de Layout

- Existem diversos tipos de layout em Android. Alguns dos mais utilizados:
 - LinearLayout — organiza os elementos em uma única direção: horizontal ou vertical.
 - ConstraintLayout — posiciona os elementos por meio de restrições, permitindo criar interfaces mais flexíveis e complexas.
 - RelativeLayout — posiciona os elementos em relação a outros elementos ou ao próprio contêiner.
 - GridLayout — organiza os elementos em uma estrutura de linhas e colunas.

LinearLayout

- O LinearLayout é um ViewGroup que organiza suas Views filhas em uma única direção, que pode ser horizontal ou vertical.
- A direção de organização é definida pelo atributo `android:orientation`.



LinearLayout

- Em uma orientação vertical, cada filho ocupa sua própria linha, formando uma coluna.
- Em uma orientação horizontal, os filhos são organizados lado a lado, formando uma única linha.
- O LinearLayout também considera as margens (margin) definidas para seus elementos filhos.
- O posicionamento dos elementos pode ser controlado por meio de atributos de gravidade e alinhamento, como `android:gravity` e `android:layout_gravity`.

LinearLayout

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="16dp"
    android:paddingRight="16dp"
    android:orientation="horizontal"
    android:gravity="center">

    <!-- Include other widget or layout tags here. These are considered
         "child views" or "children" of the linear layout -->

</LinearLayout>
```

LinearLayout

- O atributo `android:layout_weight` permite controlar como o espaço disponível é distribuído entre os elementos filhos de um `LinearLayout`, na direção definida por `android:orientation`.
- Quanto maior o valor do `layout_weight`, maior será a proporção do espaço disponível destinada àquela `View`.
- Distribuição igual do espaço:
 - Em um `LinearLayout` vertical, defina `android:layout_height="0dp"` e `android:layout_weight="1"` para cada filho.
 - Em um `LinearLayout` horizontal, defina `android:layout_width="0dp"` e `android:layout_weight="1"` para cada filho.
 - Dessa forma, os elementos filhos dividem igualmente o espaço disponível na direção do `LinearLayout`.

LinearLayout

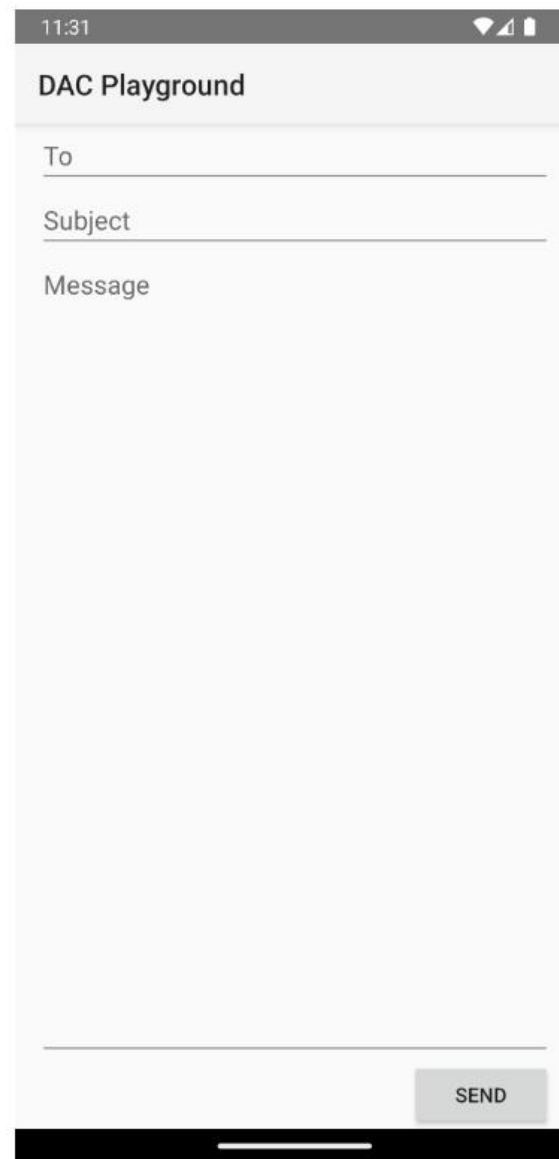
- Distribuição desigual (ponderada):
 - O `layout_weight` também permite distribuir o espaço disponível de forma proporcional entre os elementos.
 - Por exemplo, em um `LinearLayout` com três `Views`, se duas delas tiverem `android:layout_weight="1"` e a terceira não tiver peso, a terceira ocupará apenas o espaço necessário para seu conteúdo.
 - As duas `Views` com peso dividirão igualmente o espaço restante.

LinearLayout

- Distribuição desigual (ponderada):
 - O `layout_weight` também permite distribuir o espaço disponível de forma proporcional entre os elementos.
 - Por exemplo, em um `LinearLayout` com três `Views`, se duas delas tiverem `android:layout_weight="1"` e a terceira não tiver peso, a terceira ocupará apenas o espaço necessário para seu conteúdo.
 - As duas `Views` com peso dividirão igualmente o espaço restante.
 - Se houver três `Views`, sendo dois deles com `android:layout_weight="1"` e o terceiro com `android:layout_weight="2"`, o terceiro receberá o dobro do espaço disponível em relação a cada um dos outros dois.

LinearLayout

Como esse LinearLayout foi construído?



11:31

DAC Playground

To

Subject

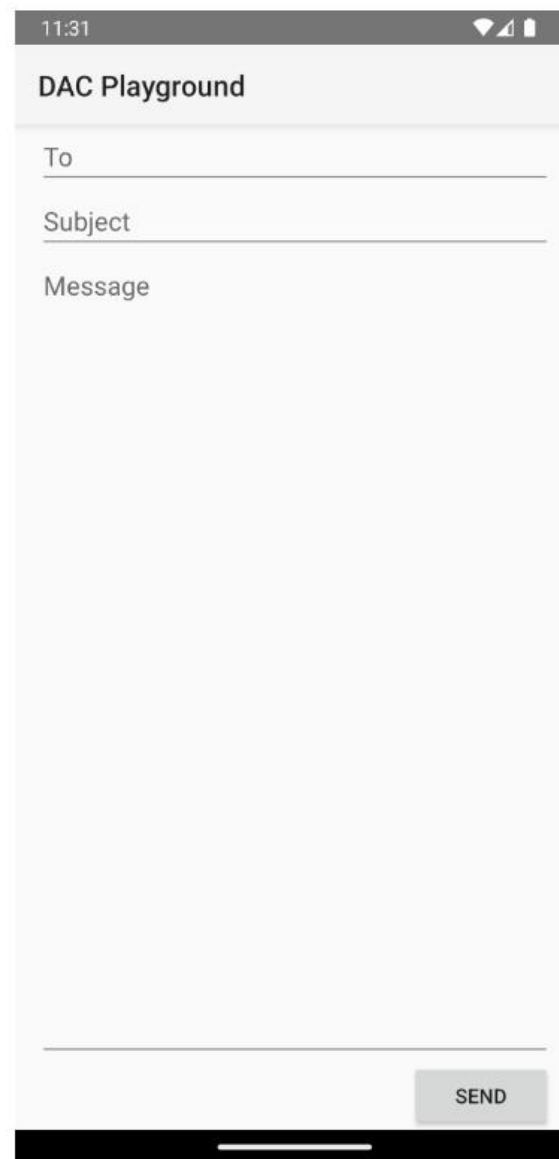
Message

SEND

Figura 2. Três campos de texto e um botão em uma `LinearLayout` orientada verticalmente.

LinearLayout

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="16dp"
    android:paddingRight="16dp"
    android:orientation="vertical" >
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/to" />
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/subject" />
    <EditText
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:gravity="top"
        android:hint="@string/message" />
    <Button
        android:layout_width="100dp"
        android:layout_height="wrap_content"
        android:layout_gravity="end"
        android:text="@string/send" />
</LinearLayout>
```



11:31

DAC Playground

To

Subject

Message

SEND

Figura 2. Três campos de texto e um botão em uma `LinearLayout` orientada verticalmente.

ConstraintLayout

- O ConstraintLayout permite criar layouts grandes e complexos com uma hierarquia de visualização mais simples e plana, evitando o aninhamento excessivo de ViewGroups.



ConstraintLayout

- É semelhante ao RelativeLayout, pois as views são posicionadas com base em relações entre outras views e o layout pai.
- O ConstraintLayout é mais flexível, pois permite estabelecer diferentes tipos de restrições entre as views. Além disso, o Layout Editor do Android Studio facilita a criação e a visualização dessas relações.

ConstraintLayout

- Para definir a posição de uma View no ConstraintLayout, é necessário estabelecer pelo menos uma restrição horizontal e uma vertical.
- Cada restrição representa uma conexão ou alinhamento da View com outra View, com o layout pai ou com uma guia (Guideline).

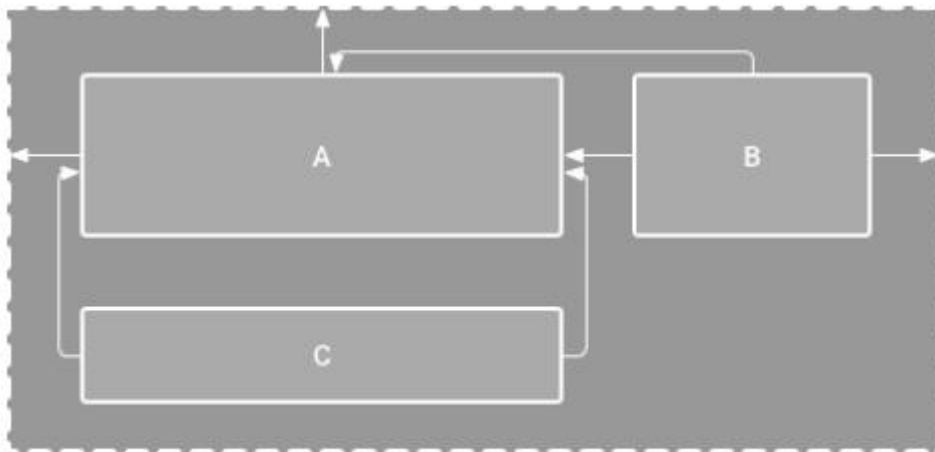


Figura 1. O editor mostra a visualização C abaixo de A, mas não tem restrição vertical.

Há algo errado com as restrições?

ConstraintLayout

- Para definir a posição de uma View no ConstraintLayout, é necessário estabelecer pelo menos uma restrição horizontal e uma vertical.
- Cada restrição representa uma conexão ou alinhamento da View com outra View, com o layout pai ou com uma guia (Guideline).

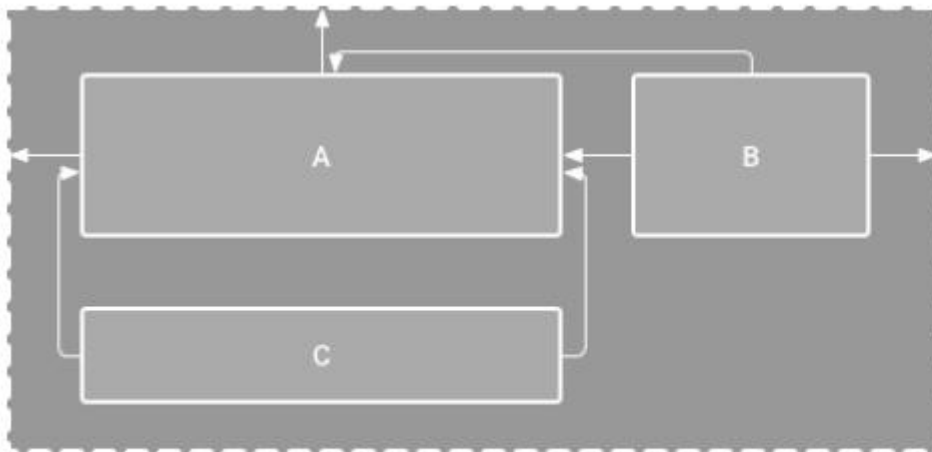


Figura 1. O editor mostra a visualização C abaixo de A, mas não tem restrição vertical.

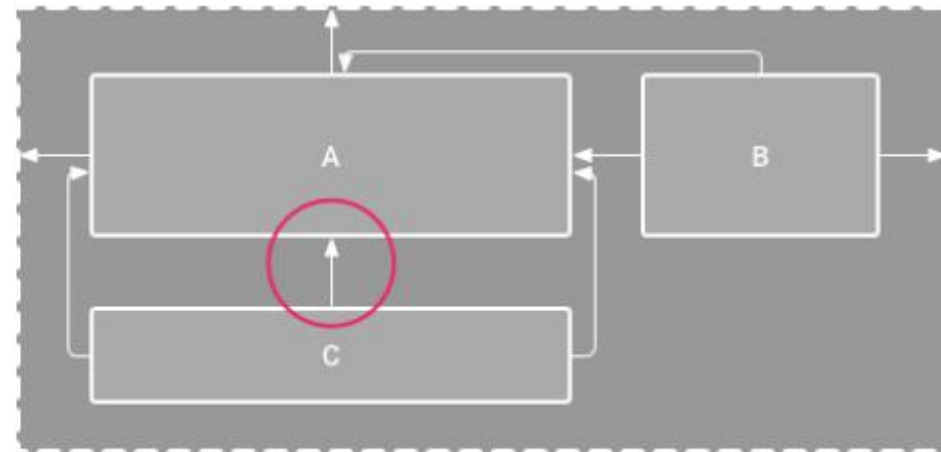
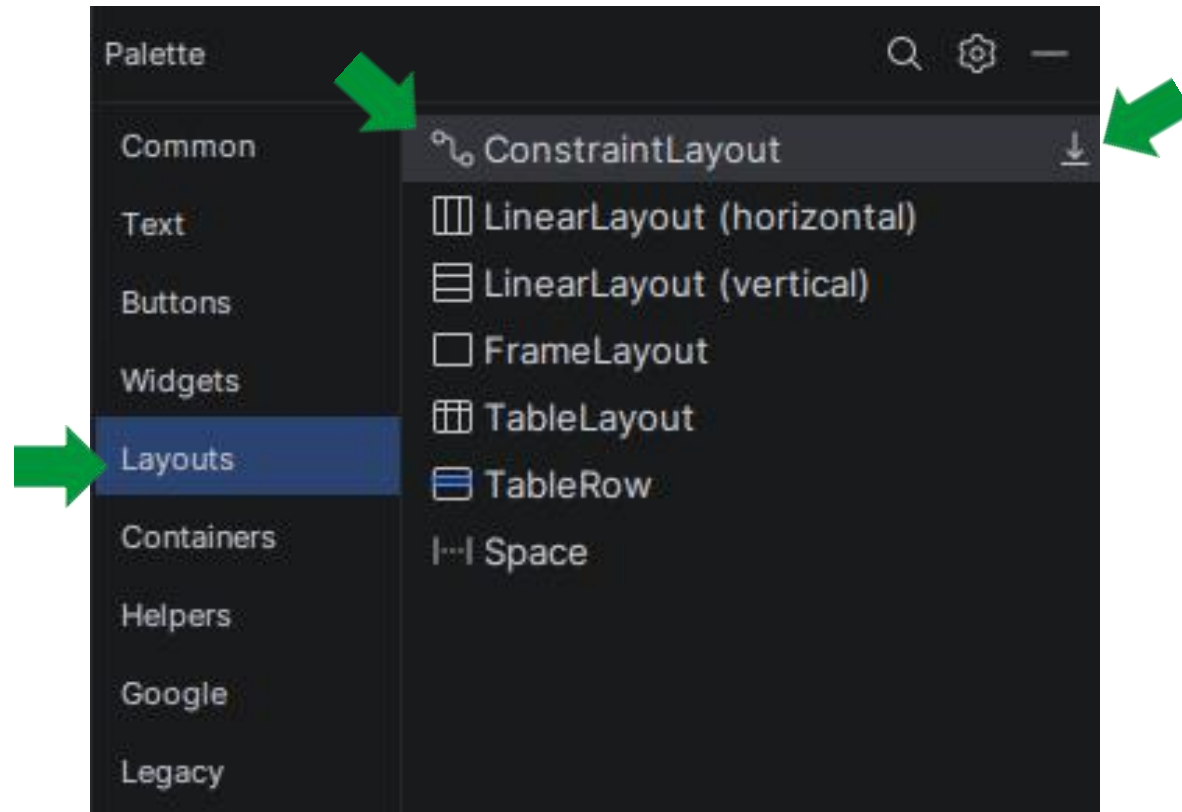


Figura 2. A visualização C está agora verticalmente restrita abaixo da visualização A.

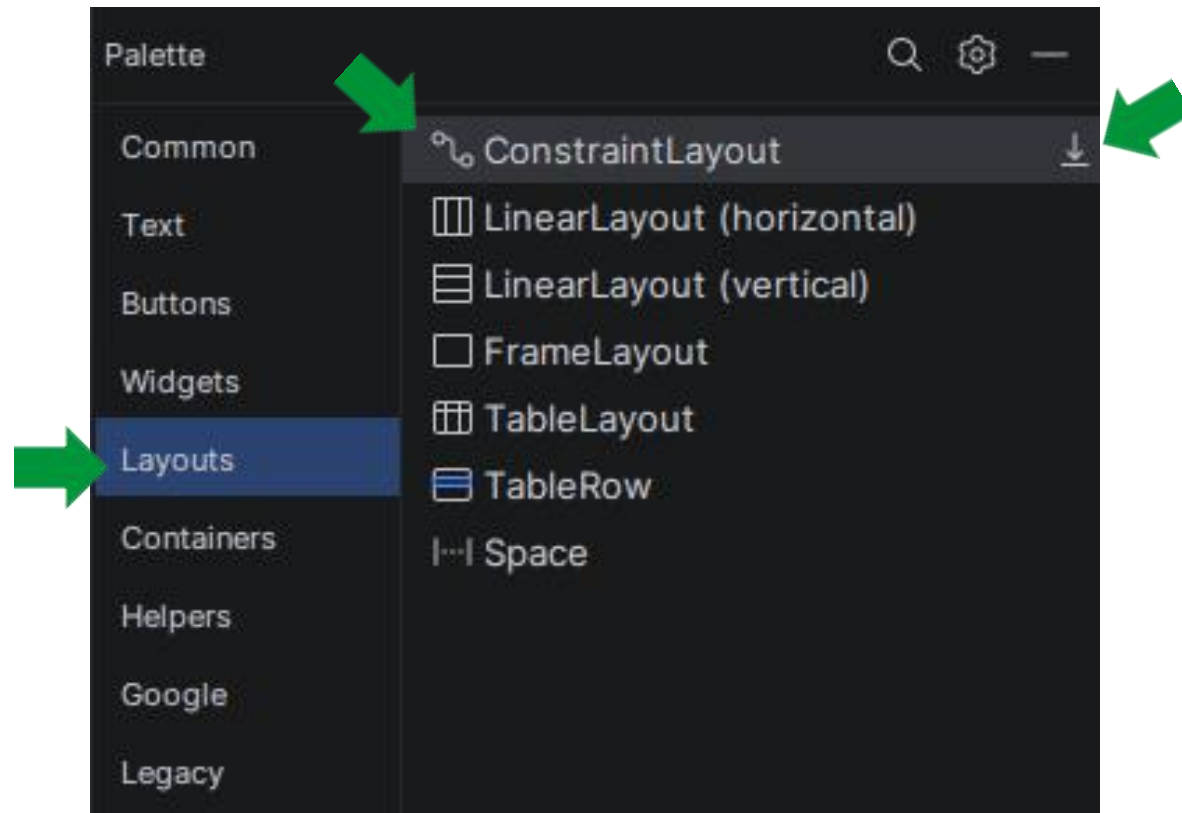
ConstraintLayout

- Adicionando ConstraintLayout a Activity:



ConstraintLayout

- Adicionando ConstraintLayout a Activity:



```
dependencies { this: DependencyHandlerScope  
    implementation(libs.androidx.constraintlayout)  
}
```

ConstraintLayout

- Retomando ao projeto de calculadora usando ConstraintLayout:



ConstraintLayout

- É possível converter um layout existente para ConstraintLayout utilizando as ferramentas disponíveis no Layout Editor do Android Studio:

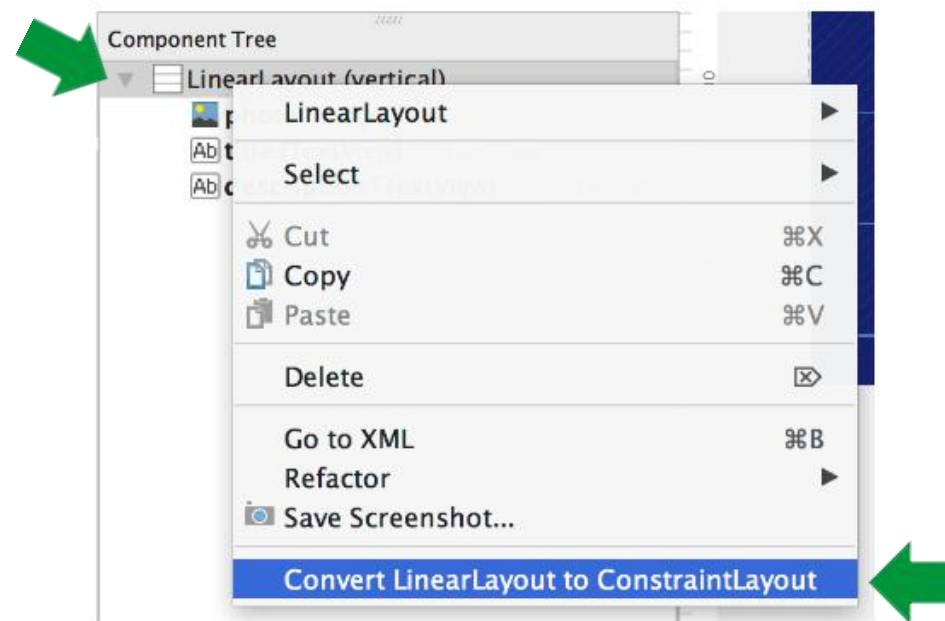


Figura 3. O menu para converter um layout para
ConstraintLayout.

RelativeLayout

- O RelativeLayout é um ViewGroup que permite posicionar suas views filhas relativamente umas às outras ou em relação ao layout pai.
- A posição de uma view pode ser definida com base em outras views irmãs (por exemplo, à esquerda, à direita, acima ou abaixo de outra view) ou em relação aos limites do layout pai (por exemplo, alinhada à parte inferior, à esquerda ou centralizada).



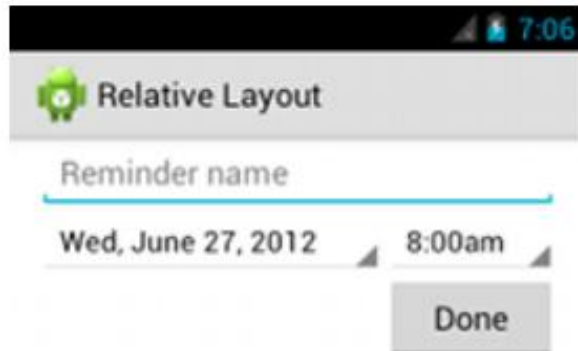
RelativeLayout

- O RelativeLayout é adequado para criar layouts relativamente complexos e pode reduzir a necessidade de layouts aninhados, contribuindo para uma hierarquia de views mais simples.
- Inicialmente, as views são posicionadas no canto superior esquerdo do layout. Por isso, é necessário definir seu posicionamento por meio das propriedades disponíveis em RelativeLayout.LayoutParams, estabelecendo relações com outras views ou com o layout pai.

RelativeLayout

- Algumas das diversas propriedades disponíveis em um RelativeLayout:
 - `android:layout_alignParentTop` → Quando definido como "true", alinha a borda superior da view à borda superior do layout pai.
 - `android:layout_below` → Posiciona a borda superior da view abaixo da view especificada pelo ID do recurso.
 - `android:layout_toRightOf` → Posiciona a borda esquerda da view à direita da view especificada pelo ID do recurso.
 - `android:layout_toEndOf` → Posiciona a View à direita da View especificada pelo seu ID.

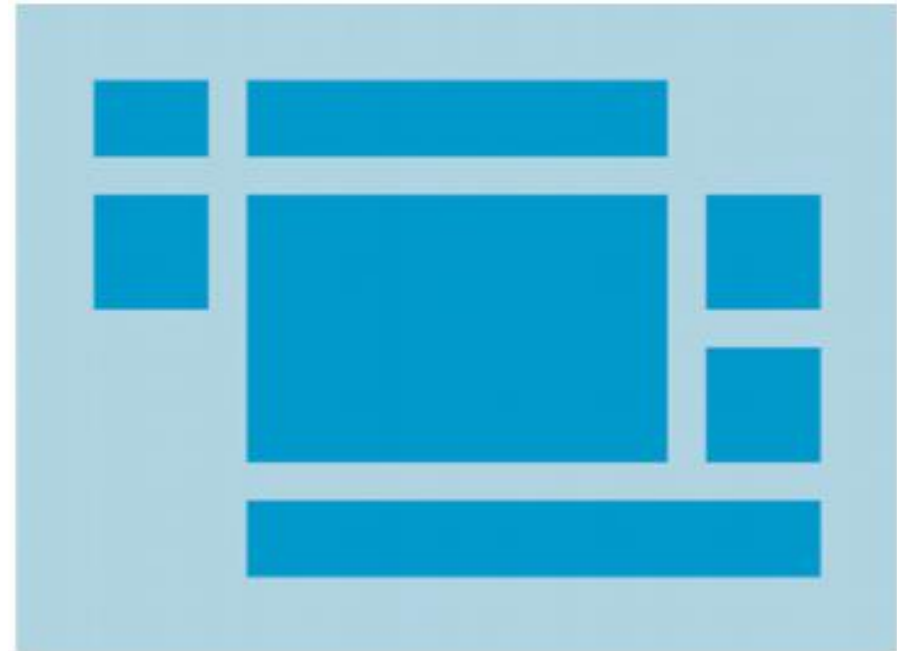
RelativeLayout



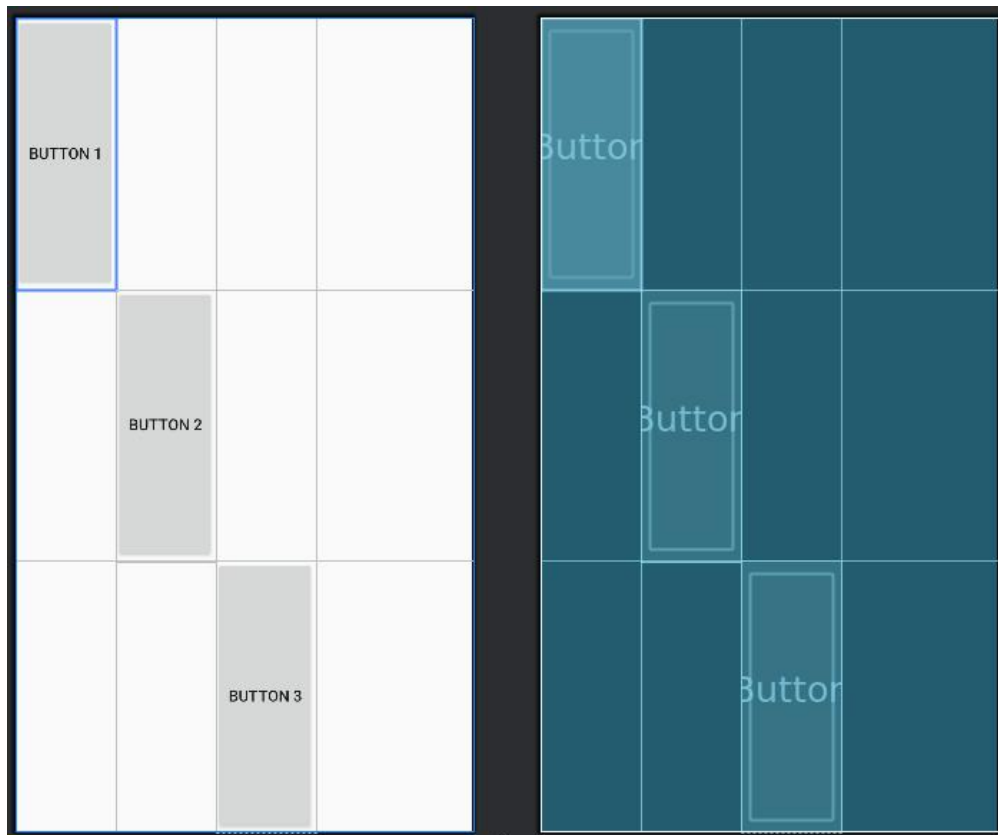
```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="16dp"
    android:paddingRight="16dp" >
    <EditText
        android:id="@+id/name"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/reminder" />
    <Spinner
        android:id="@+id/dates"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_below="@id/name"
        android:layout_alignParentLeft="true"
        android:layout_toLeftOf="@+id/times" />
    <Spinner
        android:id="@+id/times"
        android:layout_width="96dp"
        android:layout_height="wrap_content"
        android:layout_below="@id/name"
        android:layout_alignParentRight="true" />
    <Button
        android:layout_width="96dp"
        android:layout_height="wrap_content"
        android:layout_below="@id/times"
        android:layout_alignParentRight="true"
        android:text="@string/done" />
</RelativeLayout>
```

GridLayout

- O GridLayout é um ViewGroup que organiza suas views filhas em uma grade formada por linhas e colunas, permitindo definir em qual célula ou conjunto de células cada view será posicionada.



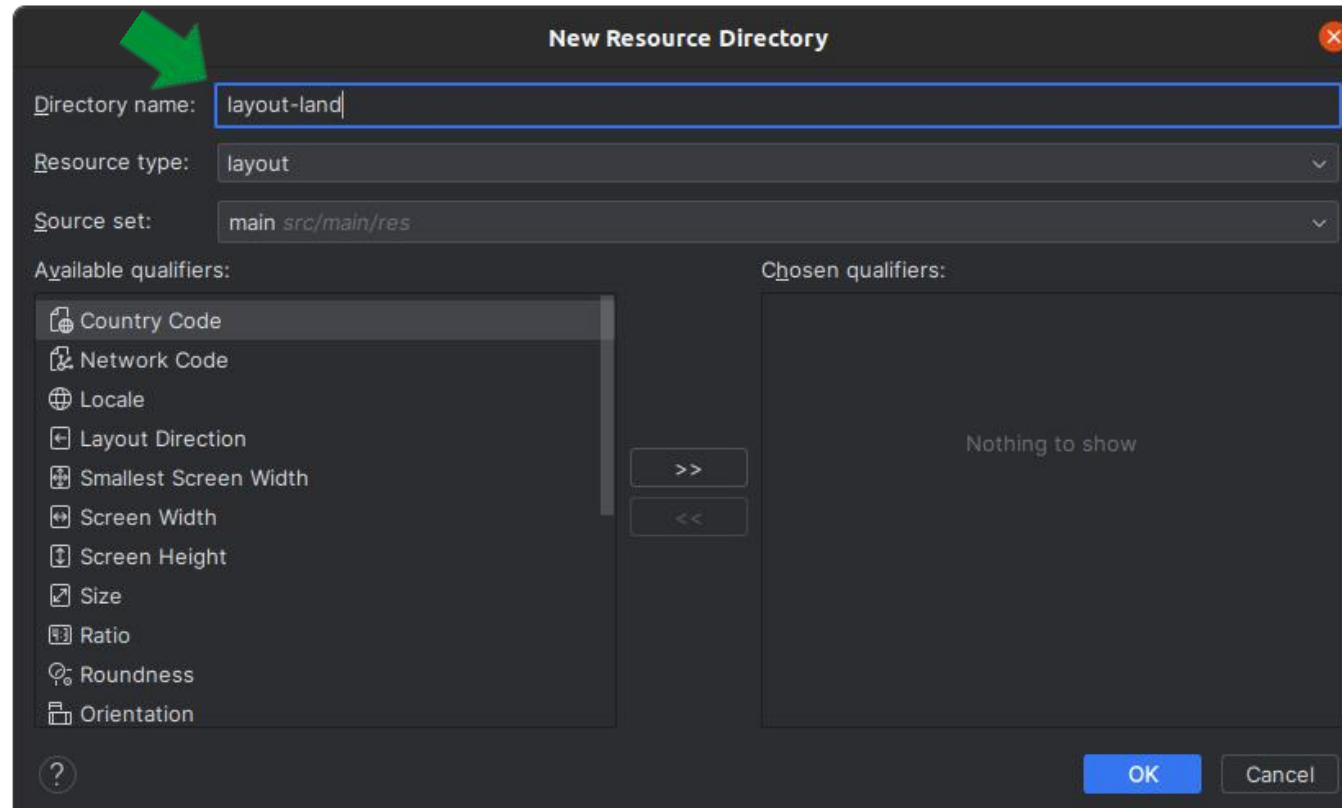
GridLayout



```
<?xml version="1.0" encoding="utf-8"?>
<GridLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:rowCount="3"
    android:columnCount="3">
    <Button
        android:text="@string/button_1"
        android:layout_width="wrap_content"
        android:layout_height="0dp"
        android:layout_row="0"
        android:layout_column="0"
        android:layout_rowWeight="1"/>
    <Button
        android:text="@string/button_2"
        android:layout_width="wrap_content"
        android:layout_height="0dp"
        android:layout_row="1"
        android:layout_column="1"
        android:layout_rowWeight="1"/>
    <Button
        android:text="@string/button_3"
        android:layout_width="wrap_content"
        android:layout_height="0dp"
        android:layout_row="2"
        android:layout_column="2"
        android:layout_rowWeight="1"/>
</GridLayout>
```

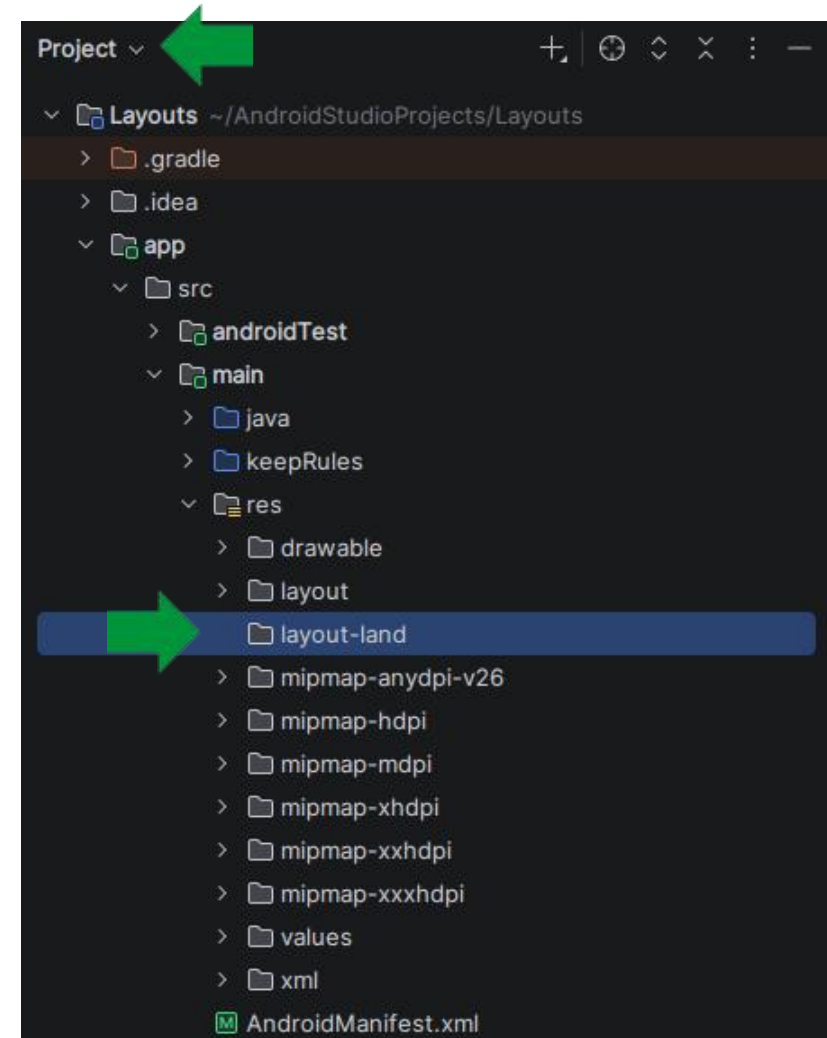
Criar layouts para diferentes orientações de tela

- new → Directory

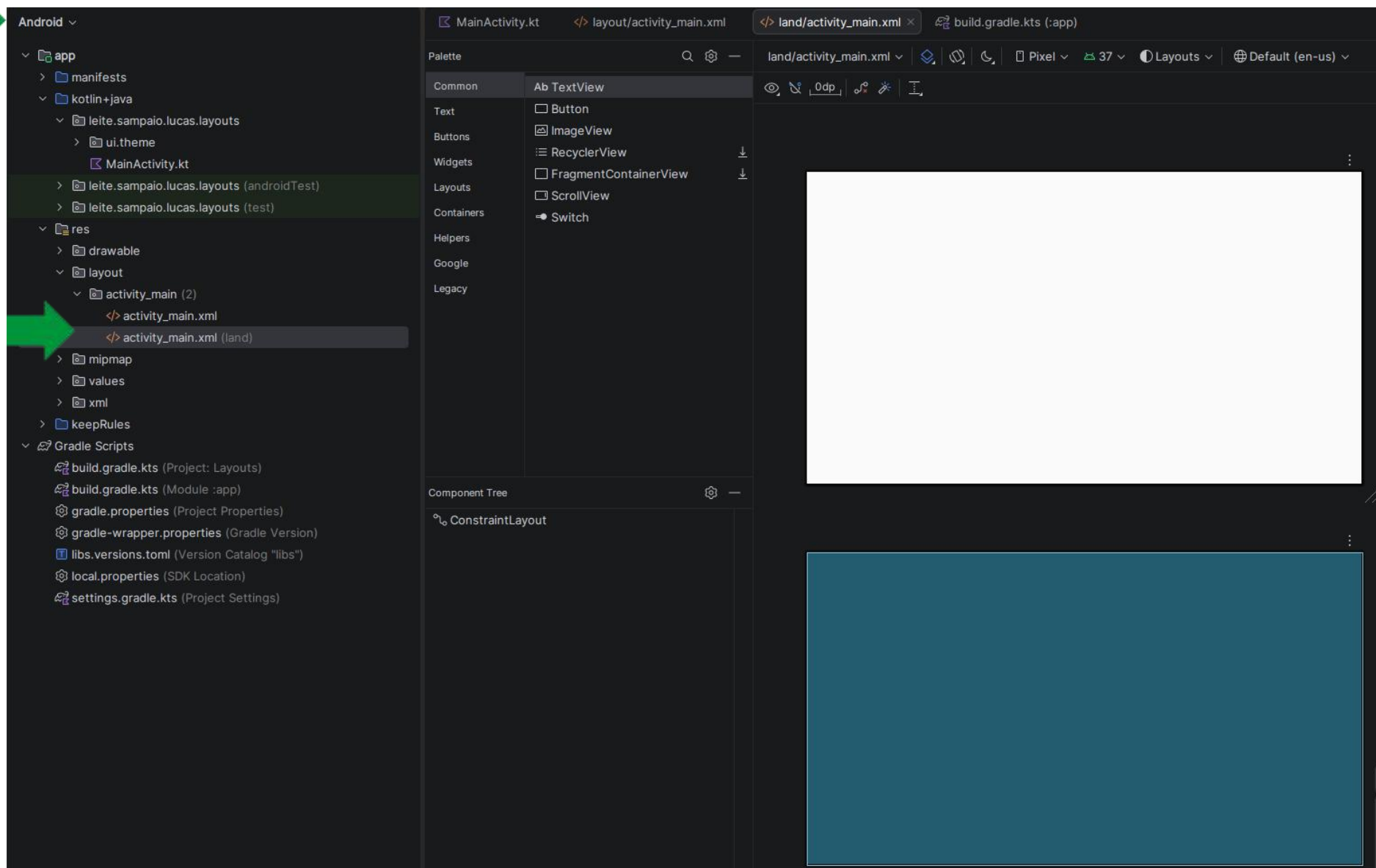


Criar layouts para diferentes orientações de tela

- Crie um arquivo de layout com o mesmo nome do layout na orientação paisagem. Ex: activity_main.xml



Criar layouts para diferentes orientações de tela



Exercícios

1. Reproduza os seguintes layouts utilizando os recursos que julgue mais adequados.



Exercícios

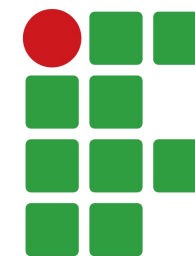
2. Pesquise sobre o componente `FrameLayout` do Android e desenvolva uma pequena interface que demonstre seu funcionamento. A aplicação deverá utilizar um `FrameLayout` como elemento principal e conter pelo menos três elementos visuais sobrepostos (por exemplo, `TextView`, `ImageView` e `Button`).

Dúvidas



PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco